tool-latency-bench

FIRST RESULTS

How much latency do tool definitions add to a voice agent?

A controlled measurement on Gemini 3.1 Flash-Lite: ~1,250 streamed requests, two experiments, two tool modes.

Pradnesh Fernandez A · November 2026

THE QUESTION

Every tool definition rides along on every request

Voice agents get about a second per turn. A bot with 60 tools sends ~5,000 extra tokens each time the user speaks. Does reading them cost time, and does *how* they are written matter?

~1 s
rough turn budget for a natural voice reply

60
real tools from voice-agent repos (Pipecat, LiveKit, τ-bench…)

~4,900
tool tokens at 60 tools, as Gemini counts them

2
questions: how many tools, and what kind

METHOD

Paired blocks, so the noise cancels

1 · One block = one message

Each of 30 user messages is sent once per condition, in random order, with a fresh nonce so nothing can be cached. Latency is compared within the block.

2 · Two experiments

exp1: 0 / 10 / 30 / 60 similar-sized tools, nested subsets.
exp2: 20 simple vs 20 medium vs 10+10 mixed tools at the same ~3,000-token budget.

3 · Two modes

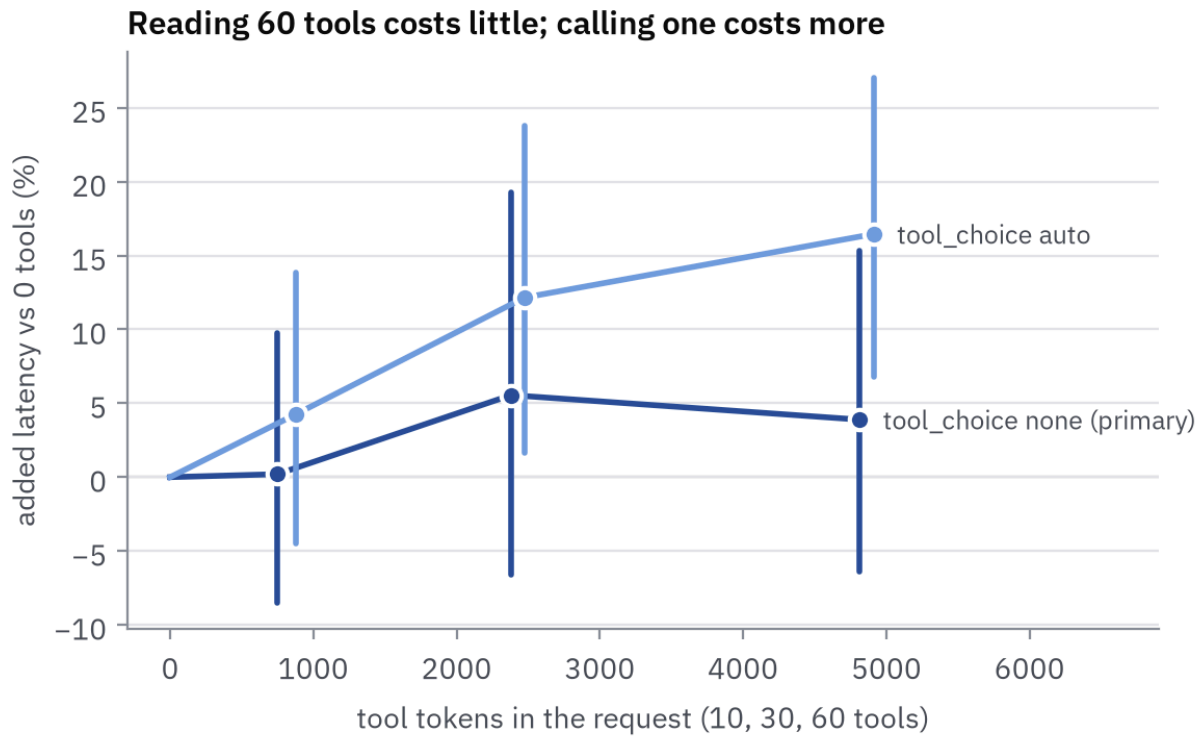
`tool_choice:` `none` measures reading the tools (primary). `auto` lets the model call them (realistic).

block 1 · “Where is my order?”	30	0	60	10	→ each condition compared with the 0-tool request
block 2 · “I want a refund”	10	60	30	0	→ order shuffled every block
… 30 blocks per run	10 ⊂ 30 ⊂ 60: the smaller sets are subsets of the larger one in each block				

Model	Reasoning	Sessions	Metric	Statistics
Gemini 3.1 Flash-Lite (paid API)	none, verified 0 thinking tokens	2 × 30 blocks per mode	time to first answer or completed tool call	cluster-robust 95% intervals

FINDING 1

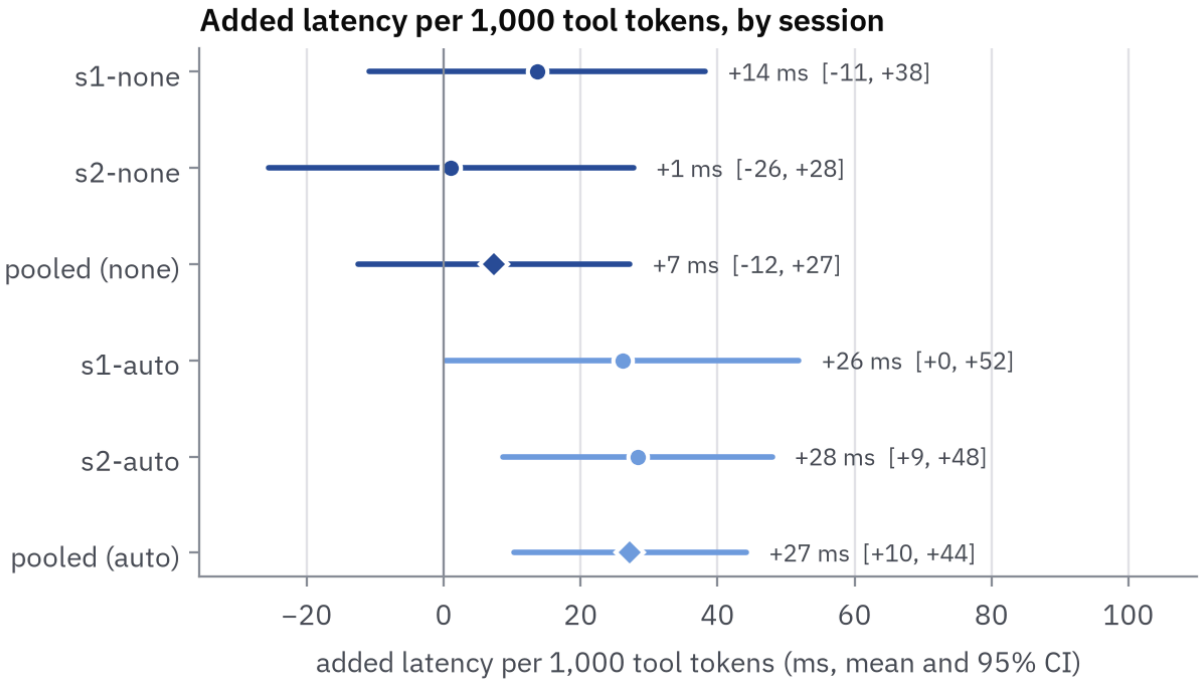
Reading tools is cheap; calling them costs more



- **Read only:** 60 tools changed latency by **+3.9%** [−6, +15] on an 820 ms baseline. At most ~130 ms; the best estimate is ~35 ms.
- **Model may call: +16.5%** [+7, +27] at 60 tools. The model calls a tool in 93% of those requests.
- The gap is mostly the time to emit the tool call, not to read the list.

FINDING 1, PER TOKEN

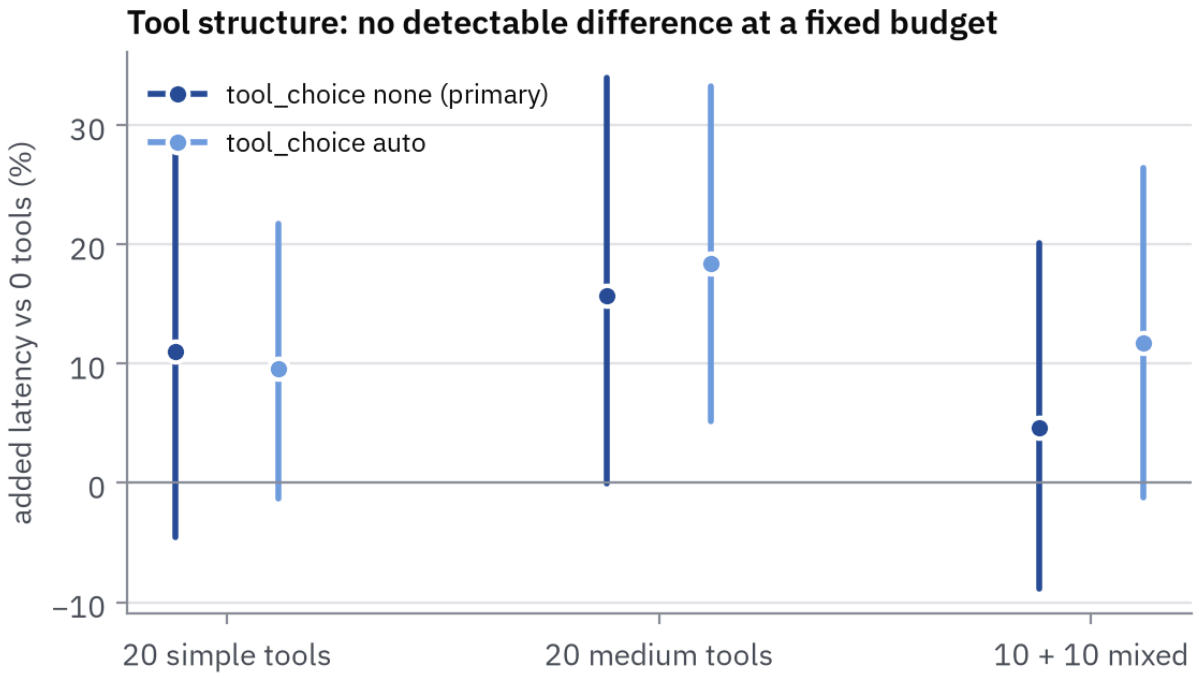
Both sessions agree



- **Read only: +7 ms** per 1,000 tool tokens [−12, +27]. The interval includes zero.
- **Model may call: +27 ms** per 1,000 tool tokens [+10, +44]. Clearly above zero, and the same in both sessions.
- One session can only detect about ±20%. Pooling sessions is what tightens these.

FINDING 2

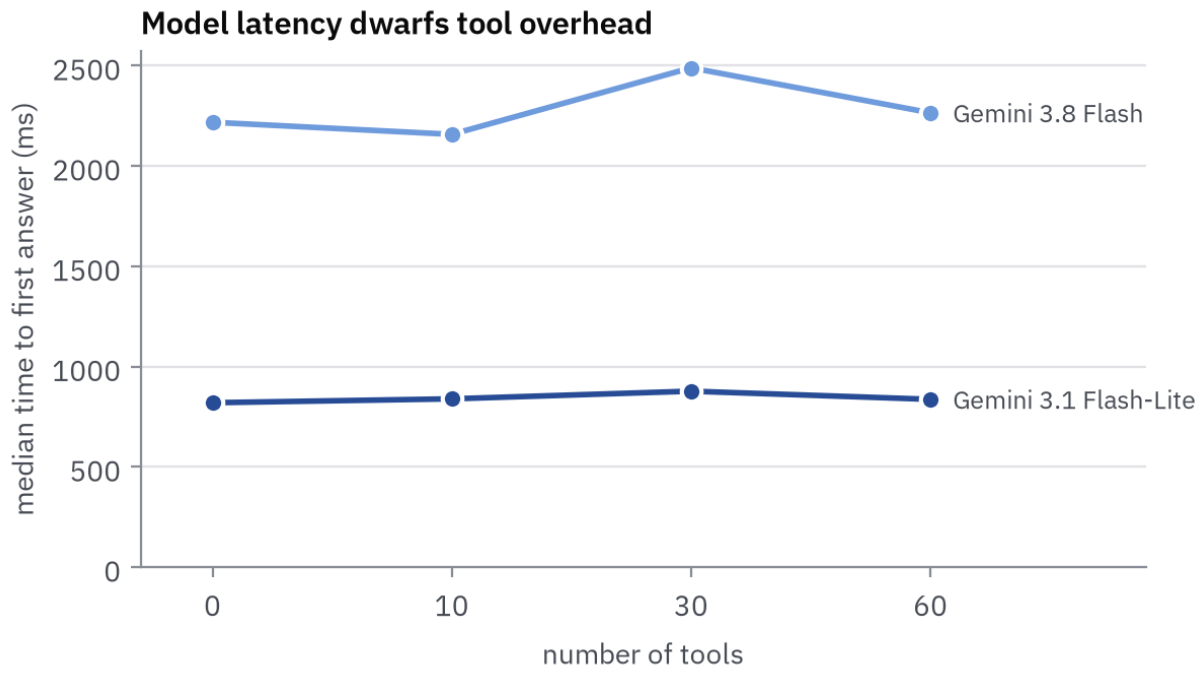
How tools are written made no detectable difference



- At a fixed ~3,000-token budget, medium vs simple tools: **+4%** [−8, +18].
- Per-token difference: +10 ms [−49, +68]. The two sessions even disagree on the sign.
- Caveat: Gemini counts medium sets as 14% more tokens than tiktoken does; the per-token rate corrects for it.

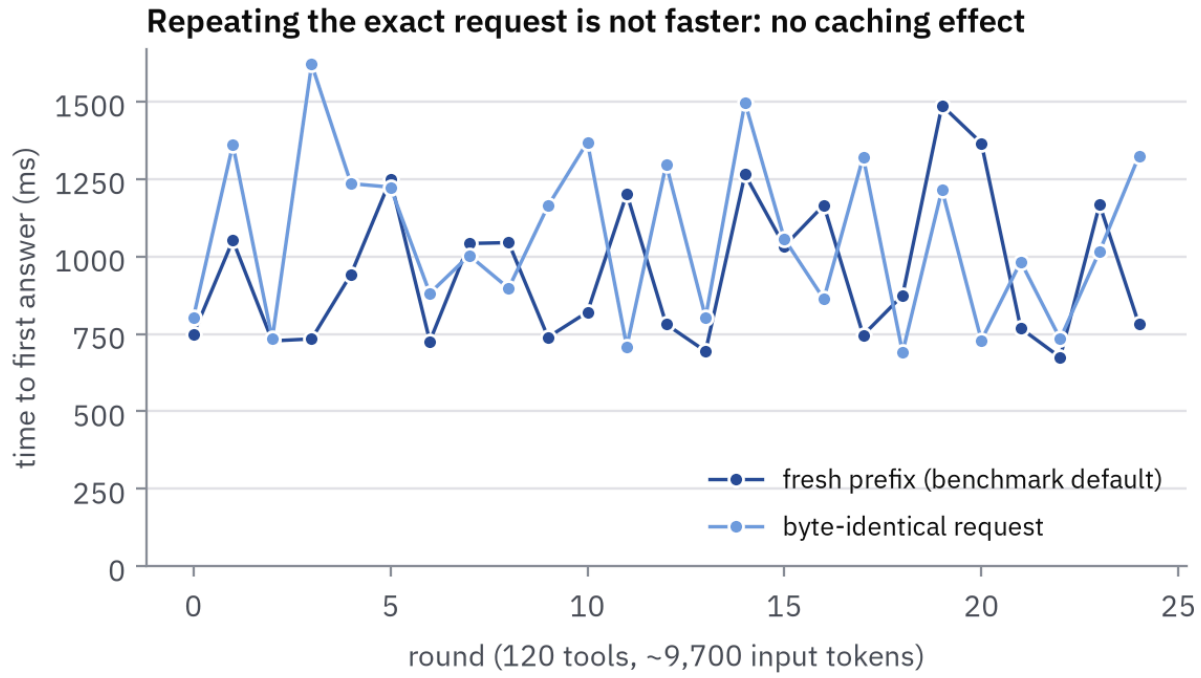
FINDING 3

The model matters about 40× more than the tools



- Gemini 3.8 Flash sits at **~2.2 s** whatever the tool count; Flash-Lite at ~0.82 s.
- On 3.8 Flash, `minimal` is rejected and none doesn't stop thinking when tools are present: the reply can run out of tokens and come back empty (17% in `auto`).
- So its numbers mix thinking with tool reading; it's reported only as this finding.

No caching effect, and a hallucinated tool call



- **Caching probe:** sending the identical 120-tool request again was not faster (ratio 1.11 [0.96, 1.29]). Gemini doesn't report cached tokens, so this is the evidence.
- **Hallucinated calls:** asked “How many reward points do I have?”, Flash-Lite invented `get_reward_points()` with *no tools declared*, in 3 of 6 tries.
- Excluding that message, order effects and drift: no change to the results.

LIMITS AND NEXT STEPS

What this doesn't show yet

Limits

- One provider, one main model
- Two sessions on the same afternoon
- OpenAI-compatible endpoint, not the native Gemini API
- Short prompts; at most 60 tools in the main runs

Next

- More sessions on other days, pooled
- OpenAI, and fast-inference hardware (Groq, Cerebras)
- 120 tools; native API vs compatibility layer
- Tool selection accuracy as the tool list grows

TAKEAWAY

Don't trim your tool list to save latency. Pick the right model, keep reasoning off, and design around tool calls.

+7 ms
per 1k tool tokens, read only

+27 ms
per 1k tool tokens, model may call

≈0
effect of tool structure

≈40×
model choice vs 60 tools